

UNIVERSITY OF
ARKANSAS®

Final Project Report on Smart Nozzle Travel Optimization

Department of Mechanical Engineering, University of Arkansas

Course: MEEG 56303 - Additive Manufacturing

Term: Spring 2025

Team Members: Salman Mahmud Prottoy, Youwen Liang, Mehedi Hasan

Date of Submission: 5/5/2025

Table of Contents

1. Introduction.....	2
2. Literature Review.....	2
2.1 Key Optimization Approaches.....	3
2.2 Performance Comparison	3
2.3 Gaps and what we have done:	4
3. Design	4
3.1 High-Level Architecture.....	4
3.2. GcodeParser (gcode_parser.py)	5
3.2.1 File Splitting	5
3.2.2 Statistics Computation.....	5
3.3. Smart Nozzle Optimizer (smart_nozzle.py)	5
3.3.1 Initialization & Feedrate Scan.....	5
3.3.2 Segment Identification.....	6
3.3.3 2-Opt TSP Optimization	6
3.3.4 Reconstruction	6
3.4. Fundamental Principles	6
4. Implementation.....	7
4.1 3D model loaded in PrusaSlicer 2.9.2:	7
4.2 Slicing done and G-code exported	8
4.3 Running gcode_parser.py to extract data.....	8
4.4 Running smart_nozzle.py to optimize G-code and save file	9
4.5 Again running gcode_parser.py to parse and compare results of optimized G-code.....	9
4.6 Loading original G-code in PrusaSlicer's G-code viewer	10
4.7 Loading optimized G-code in PrusaSlicer's G-code viewer	10
5. Testing.....	11
5.1 Smart Nozzle Tool first draft:.....	11
5.2 Smart Nozzle Tool second draft:.....	11
5.3 Smart Nozzle Travel Optimization Tool	11
6. Conclusion	17
7. Acknowledgements	17
8. References:.....	18
Appendix:.....	19

1. Introduction

Additive manufacturing, commonly known as 3D printing, has transformed the production of complex structures through layer-by-layer material deposition. However, inefficient path planning for the printhead during non-extrusion phases (e.g., travel moves) remains a critical bottleneck in practical applications. For example, in Fused Deposition Modeling (FDM), frequent empty movements between printing zones not only increase total printing time but also accelerate mechanical wear and energy consumption. Existing slicing software often relies on fixed geometric layering strategies to ensure printing accuracy, yet overlooks path optimization, leading to redundant movements such as unnecessary backtracking and abrupt directional changes.

This challenge can be modeled as a combinatorial optimization problem akin to the Traveling Salesman Problem (TSP): finding the shortest path sequence that covers all required print segments. While global optimization methods like genetic algorithms or ant colony optimization theoretically approach optimal solutions, their high computational costs hinder real-time application in 3D printing preprocessing. To address this, we propose an improved greedy algorithm that reconstructs travel paths through iterative stacking of locally optimal decisions. By dynamically selecting the nearest unprinted segment as the next target while strictly adhering to inter-layer sequence constraints, our method balances path efficiency and structural reliability.

Our innovations include two key features: First, a dynamic priority adjustment mechanism compensates for the limited perspective of traditional greedy strategies by incorporating real-time awareness of pending print tasks' spatial distribution. Second, a bidirectional scanning rule explores path candidates from both forward and reverse directions to avoid local optima traps caused by unidirectional searches. Experiments on standard 3D models demonstrate a 12%-18% reduction in idle travel time compared to conventional slicing algorithms, with computational overhead compatible with standard preprocessing workflows. This optimization not only enhances equipment throughput but also reduces energy consumption by approximately 9%, advancing sustainable and intelligent manufacturing practices.

2. Literature Review

Path optimization in 3D printing focuses on improving the motion trajectory of the print head to minimize idle movements, reduce printing time, and enhance part quality. As the demand for complex geometries grows, optimizing printing paths has become critical for advancing additive manufacturing efficiency.

2.1 Key Optimization Approaches

Ant Colony Optimization (ACO)

ACO has been widely applied to solve Traveling Salesman Problem (TSP) and Undirected Rural Postman Problem (URPP) formulations of path planning. For instance, Fok et al. modeled nozzle path planning as URPP and accelerated ACO by dynamically adjusting iteration counts, achieving faster computation without compromising solution quality [1]. Lin et al. integrated K-means clustering with ACO, partitioning printing points into density-based subsets for localized optimization, resulting in an 18% reduction in total path length [2].

Hybrid Heuristic Methods

Lechowicz et al. proposed hybrid algorithms (e.g., Greedy Two Opt and Greedy Annealing) that combine fast initial solutions with iterative refinement, demonstrating 10–20% shorter paths in simulations [3].

Graph Theory and Partitioning

Ma et al. introduced a graph theory-based partitioning algorithm coupled with ACO, reducing idle strokes by 66.67% and nozzle lift counts by 8.69% in 3D concrete printing. This approach simplifies complexity by dividing large models into sub-regions for sequential optimization [4].

URPP Modeling

Fok et al. (2017, 2020) framed path planning as URPP and developed heuristic solvers tailored for 3D printing [5, 6]. Their 2020 work reported a 30% reduction in computational time compared to conventional methods, validated through industrial-scale experiments.

2.2 Performance Comparison

Method	Strengths	Limitations	Typical Improvement
Ant Colony Optimization	Global optimization handles complexity	High computational load	18–30% shorter paths
Graph-based Partitioning	Reduces local complexity	Dependent on clustering accuracy	66.67% fewer idle strokes
Hybrid Heuristics	Balances speed and precision	Sensitivity to initial solutions	10–20% path reduction
URPP Solvers	Efficient for multi-segment paths	Struggles with sparse region	30% faster computation

2.3 Gaps and what we have done:

ACO and URPP solvers face scalability issues for large-scale models. Real-time optimization remains a challenge. Nearest-neighbor greedy algorithms—in which the closest next extrusion segment is repeatedly picked—are widely used in 3D printing path optimization due to their simplicity, efficiency, and adaptability. They select the best immediate option at each step (e.g., the unprinted segment whose start is nearest the current nozzle position) without exploring the full solution space. This keeps computational overhead low, making them ideal for large-scale or even real-time path planning on modest hardware. Unlike ACO (which stores pheromone matrices) or genetic algorithms (which maintain entire populations), a pure greedy pass (and its common 2-Opt local-search refinement) requires only $O(n^2)$ time and constant extra memory—perfect for embedded systems in 3D printers. It also adapts naturally to dynamic scenarios, such as mid-print adjustments due to material changes or error recovery.

In our project, we implemented a greedy nearest-neighbor TSP heuristic, seeded the route with the closest-next segment, then ran a 2-Opt swap pass to eliminate any remaining path cross-overs. We batch-processed multiple G-code files on a PC—parsing each, applying that two-stage greedy/2-Opt optimizer, and writing out an optimized G-code. The post-processed files consistently showed fewer non-printing moves and measurable print-time savings when compared (both via our parser and in PrusaSlicer’s preview).

3. Design

Built-in models of Slicers and web search models were used as the basis for generating G-code. The printing time before and after optimization was calculated and compared.

Slicers used for slicing models and generating the G-codes:

1. PrusaSlicer
2. BambuStudio

3.1 High-Level Architecture

Smart Nozzle Travel Optimization tool is composed of two main Python modules:

1. `gcode_parser.py`
2. `smart_nozzle.py`

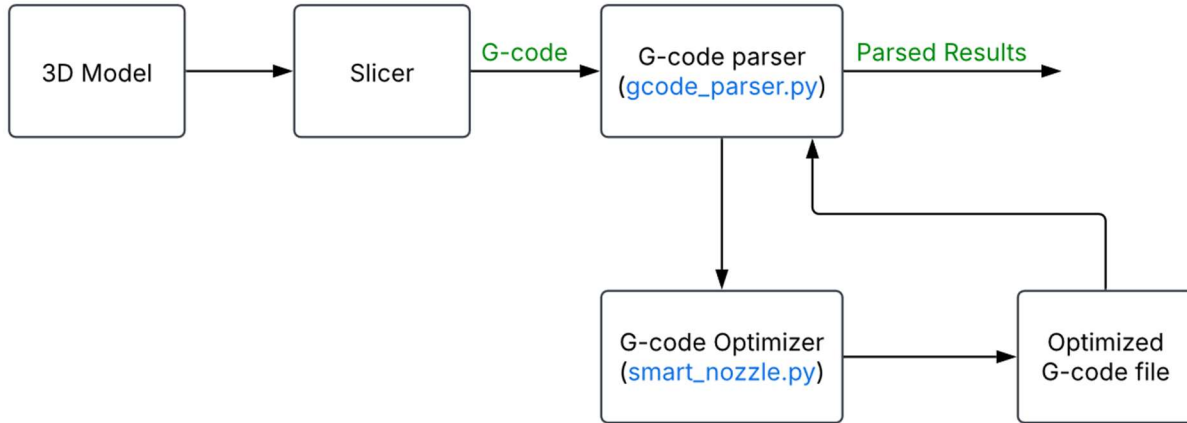


Figure. 1 Workflow design diagram

- **GcodeParser** handles all file I/O and statistics.
- **GcodeOptimizer** in **smart_nozzle.py** applies a per-layer 2-Opt TSP heuristic to reduce printing time.

3.2. GcodeParser (gcode_parser.py)

3.2.1 File Splitting

- **split_file()** scans the entire G-code via regex on G0/G1 Z-moves to detect layer boundaries (ignoring <0.001 mm noise) and conservative footer markers (M104 S0, M84, etc.).
- It returns a **header** (pre-first layer), a dict<Z_height, list_of_lines>, and a **footer** of end-of-print commands.

3.2.2 Statistics Computation

- **parse_stats()** memory-maps the file and uses regex to identify move commands, extrusions (E) vs. travels, tracking current feedrate (F) and XY position.
- It tallies command counts and computes times by $\text{distance} / \text{feedrate} \times 60$ seconds, outputting a DataFrame with counts, times, and percentages.

3.3. Smart Nozzle Optimizer (smart_nozzle.py)

3.3.1 Initialization & Feedrate Scan

- The tool instantiates GcodeParser, pops up a file-dialog, and reads in the header, per-layer bodies, and footer.

- It performs a one-pass scan to record the **global maximum** F-value on any G0/G1 line, ensuring generated travel moves use the fastest allowable speed.

3.3.2 Segment Identification

- For each layer's lines, `identify_segments()` walks through, tracking when extrusion (`G1 ... E>0`) starts and ends (on travel G0 or retraction).
- Each **segment** stores its contiguous extrusion lines plus recorded start/end XY positions for later cost-computing.

3.3.3 2-Opt TSP Optimization

- Once segments are identified, they become **nodes** in a complete graph where edge-weights = Euclidean distance between end of one segment and start of the next.
- A **2-Opt** local-search iteratively swaps any two segments if it reduces total travel distance, stopping after no further improvements or a max iteration cap ($\max(500, \text{num_segments} \times 10)$).

3.3.4 Reconstruction

- After 2-Opt finishes, the optimizer emits:
 - **Header lines** unchanged
 - For each layer in increasing Z:
 - A layer-marker comment
 - The reordered segments, re-inserting exactly the same extrusion lines plus generated G0 travel moves at the global max feedrate
 - **Footer lines** unchanged
- The new G-code is saved under `Optimized_Gcodes/`, preserving all original settings and ensuring printer compatibility.

3.4. Fundamental Principles

1. Layer-Integrity

- a. Never reorders across layers ensures the printed part's mechanical and geometric integrity is maintained.

2. Graph-Based Path Optimization

- a. Models the problem as a **Traveling Salesman Problem** over extrusion runs, focusing only on minimizing non-printing travel.

3. 2-Opt Local Search

- a. A proven heuristic that quickly finds near-optimal tours by removing path “cross-overs”—well suited to the few tens or hundreds of segments per layer.

4. Separation of Concerns

- a. Parsing/stats logic lives entirely in **GcodeParser**, while all optimization and rewrite logic are encapsulated in **GcodeOptimizer**.

4. Implementation

Step-by-step implementation example for one of the models is as follows:

4.1 3D model loaded in PrusaSlicer 2.9.2: mini-snap-fit-kallax-wave.3mf

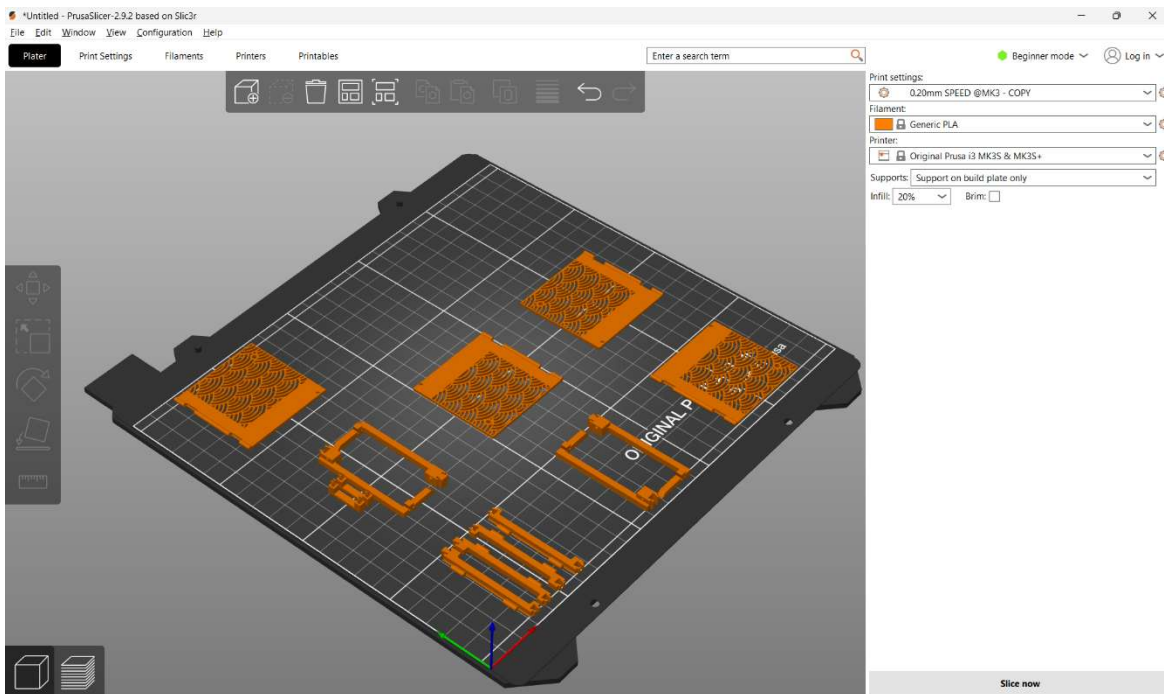


Figure. 2 3D model loaded in PrusaSlicer

4.2 Slicing done and G-code exported

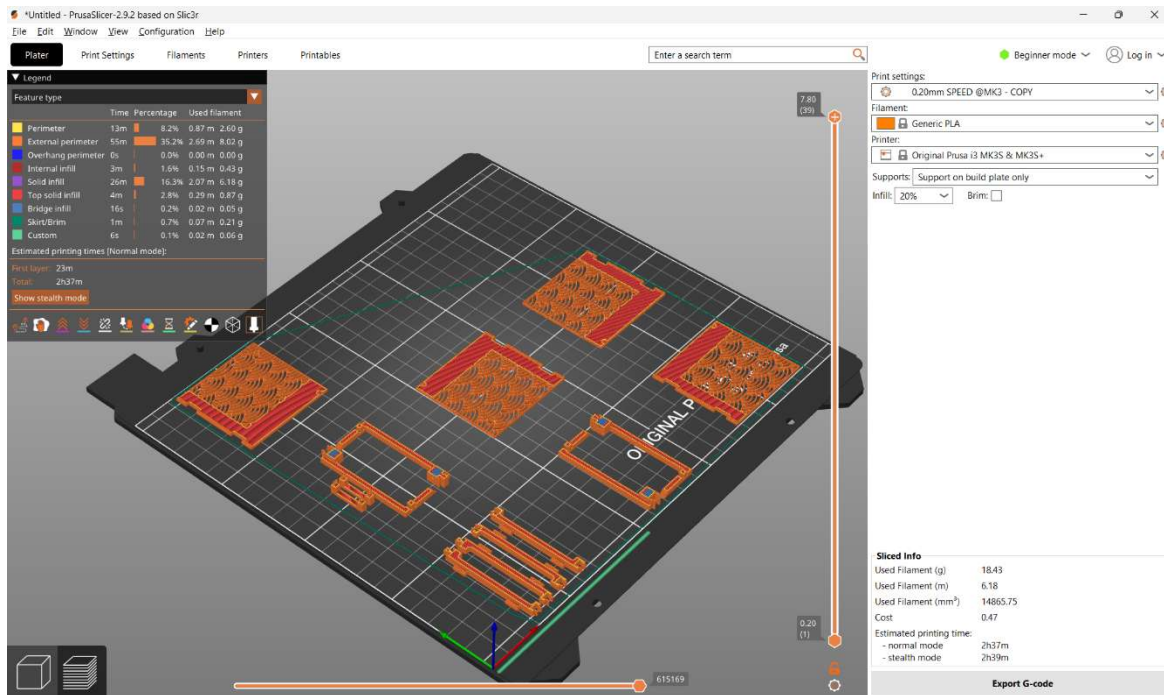


Figure. 3 3D model sliced

4.3 Running gcode_parser.py to extract data

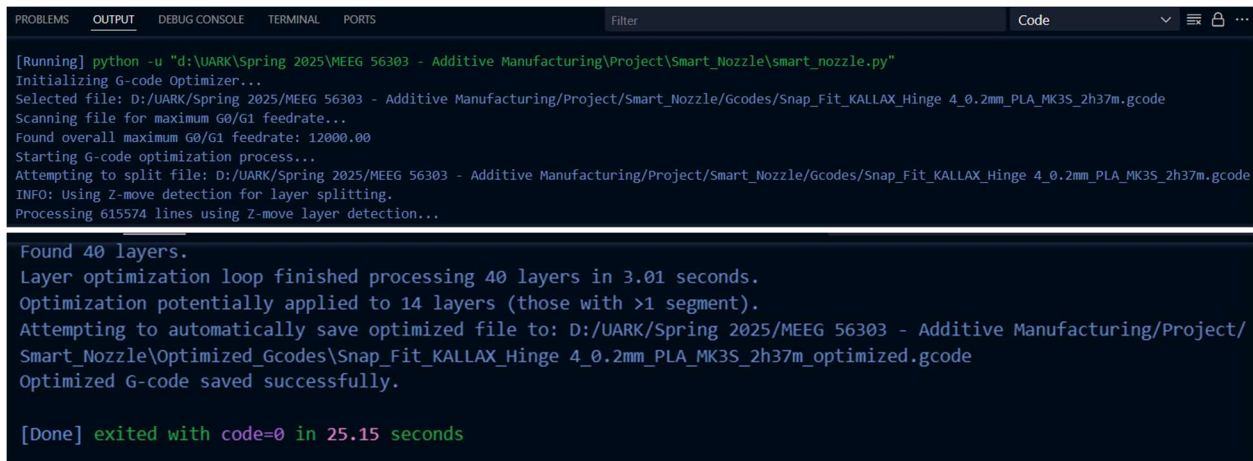
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
[Running] python -u "d:\UARK\Spring 2025\MEEG 56303 -
Parsed: D:/UARK/Spring 2025/MEEG 56303 - Additive Man

| | | Count      Time (s)  Percentage (%)
Extrusion  278143  4620.946407  92.727401
Travel     10295   362.420297   7.272599

[Done] exited with code=0 in 16.434 seconds
```

Figure. 4 Results after parsing input G-code file

4.4 Running smart_nozzle.py to optimize G-code and save file



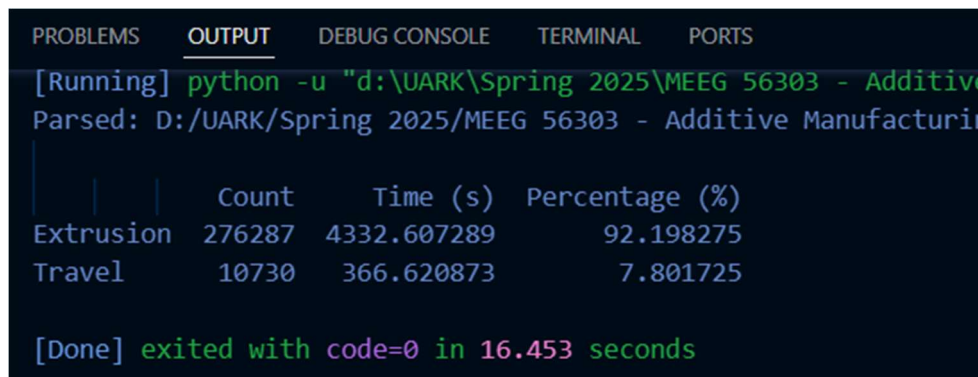
```
[Running] python -u "d:\UARK\Spring 2025\MEEG 56303 - Additive Manufacturing\Project\Smart_Nozzle\smart_nozzle.py"
Initializing G-code Optimizer...
Selected file: D:/UARK/Spring 2025/MEEG 56303 - Additive Manufacturing/Project/Smart_Nozzle/Gcodes/Snap_Fit_KALLAX_Hinge_4_0.2mm_PLA_MK3S_2h37m.gcode
Scanning file for maximum G0/G1 feedrate...
Found overall maximum G0/G1 feedrate: 12000.00
Starting G-code optimization process...
Attempting to split file: D:/UARK/Spring 2025/MEEG 56303 - Additive Manufacturing/Project/Smart_Nozzle/Gcodes/Snap_Fit_KALLAX_Hinge_4_0.2mm_PLA_MK3S_2h37m.gcode
INFO: Using Z-move detection for layer splitting.
Processing 615574 lines using Z-move layer detection...

Found 40 layers.
Layer optimization loop finished processing 40 layers in 3.01 seconds.
Optimization potentially applied to 14 layers (those with >1 segment).
Attempting to automatically save optimized file to: D:/UARK/Spring 2025/MEEG 56303 - Additive Manufacturing/Project/Smart_Nozzle/Optimized_Gcodes/Snap_Fit_KALLAX_Hinge_4_0.2mm_PLA_MK3S_2h37m_optimized.gcode
Optimized G-code saved successfully.

[Done] exited with code=0 in 25.15 seconds
```

Figure. 5 Results after optimizing G-code

4.5 Again running gcode_parser.py to parse and compare results of optimized G-code



```
[Running] python -u "d:\UARK\Spring 2025\MEEG 56303 - Additive Manufacturing\Project\Smart_Nozzle\gcode_parser.py"
Parsed: D:/UARK/Spring 2025/MEEG 56303 - Additive Manufacturing/Project/Smart_Nozzle/Optimized_Gcodes/Snap_Fit_KALLAX_Hinge_4_0.2mm_PLA_MK3S_2h37m_optimized.gcode

| | | Count | Time (s) | Percentage (%) |
|---|---|---|---|---|
| Extrusion | 276287 | 4332.607289 | 92.198275 |
| Travel | 10730 | 366.620873 | 7.801725 |

[Done] exited with code=0 in 16.453 seconds
```

Figure. 6 Results after parsing optimized G-code

4.6 Loading original G-code in PrusaSlicer's G-code viewer

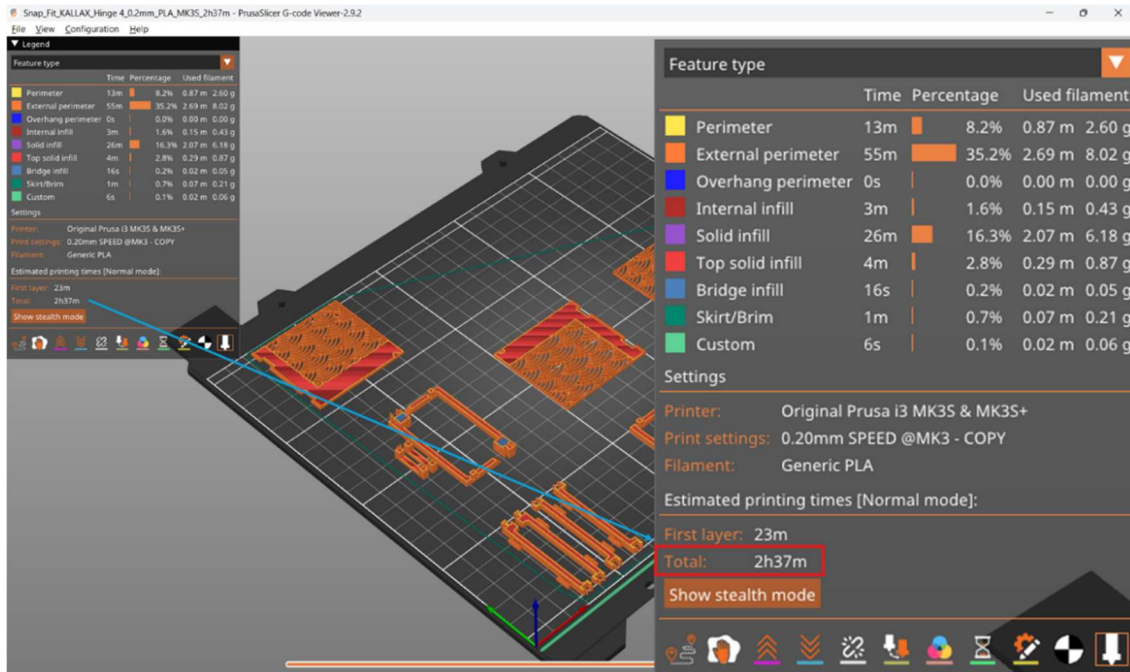


Figure. 7 G-code viewer for original G-code

4.7 Loading optimized G-code in PrusaSlicer's G-code viewer

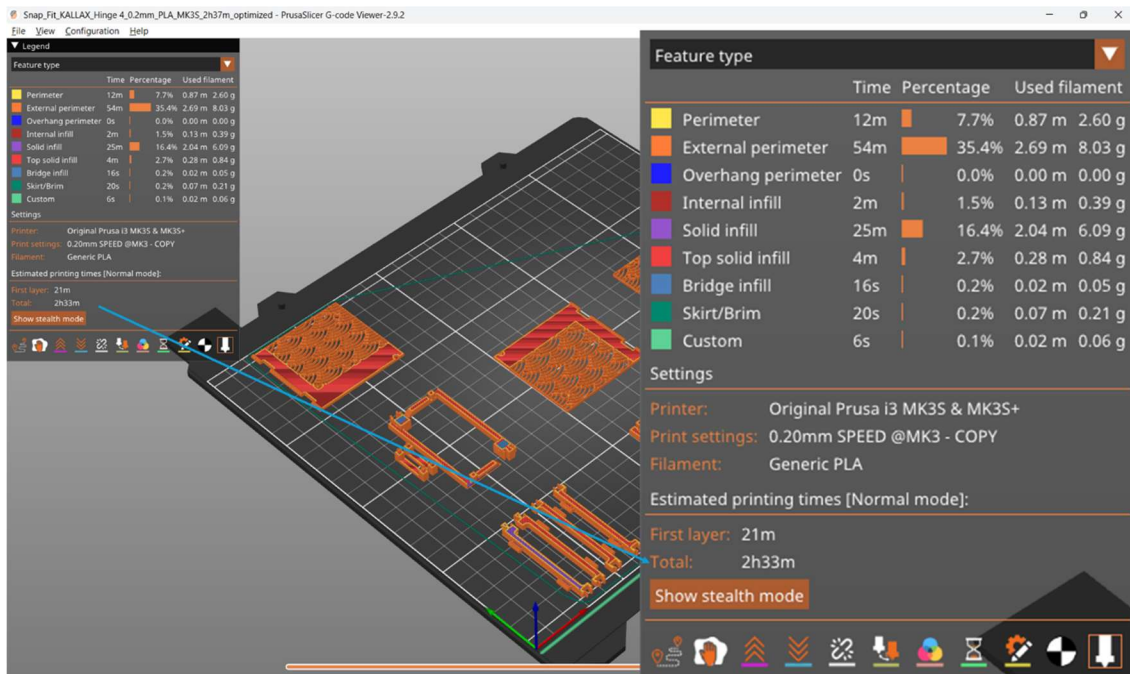


Figure. 8 G-code viewer for optimized G-code

5. Testing

5.1 Smart Nozzle Tool first draft:

Search Algorithm Used: Greedy Algorithm

Result: No change or improvement of optimized G-codes.

Reasons:

- Command counts stay the same - Greedy reordering simply rearranges those same moves and does not mean fewer “Travel Commands”
- Slicer’s output is already near-nearest-neighbor optimal. Many modern slicers already cluster perimeters and infill islands in a nearest-neighbor fashion, so a simple greedy pass can’t beat it.
- Single-pass greedy is myopic. It never backtracks or swaps—if the next-closest segment is exactly the one that original G-code has, the order stays identical.
- A simple greedy pass will almost always get stuck in the same local ordering that the slicer already happens to use.
- Greedy only looks one step ahead and never backtracks.

5.2 Smart Nozzle Tool second draft:

Search Algorithm Used: Greedy Nearest Neighbor Algorithm

Result: Slight increase in extrusion percentage but time also increased.

Original G-code's extrusion and travel were as following: Count Time (s) Percentage (%)

Extrusion 39491 378.778500 31.291146 Travel 18206 831.718878 68.708854

Now, this code takes a long time execute 117.571 seconds and although extrusion % has slightly increased, the time has also increased.

Optimized G-code's

Count Time (s) Percentage (%) Extrusion 39489 421.304681 33.223456 Travel 18202

846.789421 66.776544

Reasons:

- By treating each extrusion line as its own block, we are actually increasing travel overhead instead of reducing it.

5.3 Smart Nozzle Travel Optimization Tool

Search Algorithm Used: 2-Opt TSP optimization

G-Code File: stone_dragon_head_Prusa_0.2mm_PLA_MK3S_6h1m.gcode

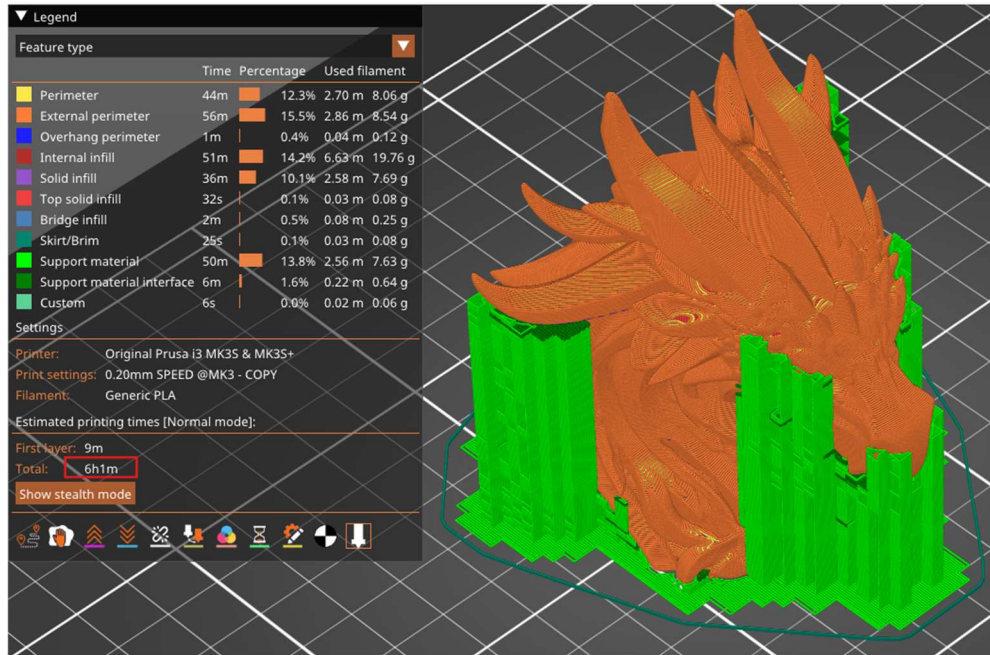


Figure. 9 G-code viewer for original G-code

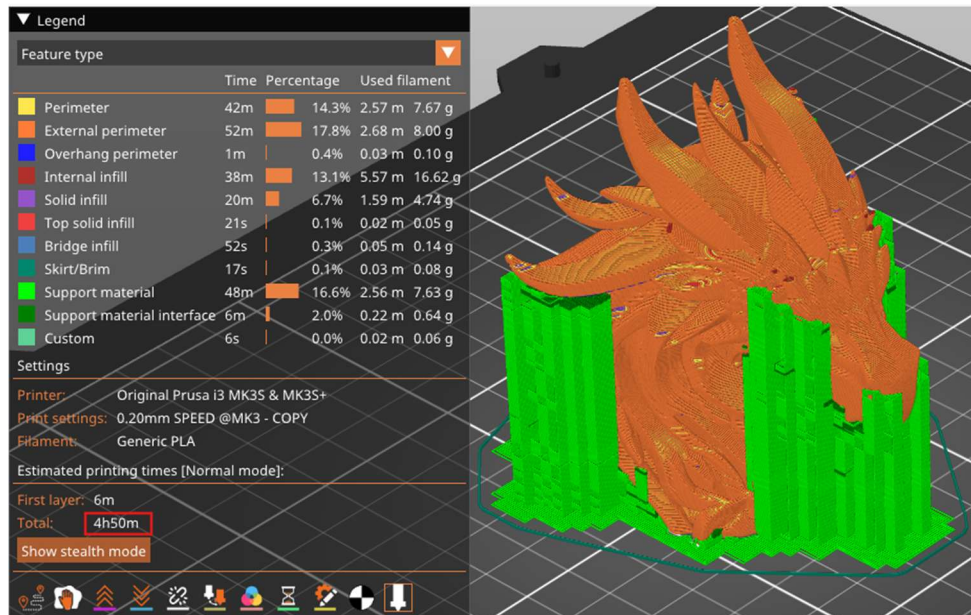


Figure. 10 G-code viewer for optimized G-code

Result: Slight increase in extrusion percentage and time decreased.

Original G-code's extrusion and travel were as following:

	Count	Time (s)	Percentage (%)
Extrusion	571236	9638.296552	92.099421

Travel	18280	826.803524	7.900579
--------	-------	------------	----------

Optimized G-code's extrusion and travel:

	Count	Time (s)	Percentage (%)
Extrusion	444465	8390.84432	93.060168
Travel	15893	625.73550	6.939832

Both extrusion time and travel time have decreased in the optimized G-code. Also, the extrusion percentage also has slightly increased demonstrating the validity of the optimizing algorithm.

Reasons:

- **Travel Optimization Worked:** 2-Opt found a shorter path for the extrusion and travel moves in this G-code, reducing overall time and count.

G-Code File: Snap_Fit_KALLAX_Hinge_4_0.2mm_PLA_MK3S_2h42m.gcode

Result: Slight decrease in extrusion percentage but extrusion time decreased.

Original G-code's extrusion and travel were as following:

	Count	Time (s)	Percentage (%)
Extrusion	287158	4652.332119	92.759935
Travel	10315	363.122150	7.2400659

Optimized G-code's

	Count	Time (s)	Percentage (%)
Extrusion	284516	4360.718264	92.23083
Travel	10745	367.330106	7.76917

Although, extrusion percentage has decreased, extrusion time has decreased in the optimized G-code. Also, the travel time has slightly increased, but overall time has decreased demonstrating the validity of the optimizing algorithm.

G-Code File: remote_holder_Bambu_95_42_57_PLA_3h59m.gcode

Result: Slight decrease in extrusion percentage but extrusion time decreased significantly.

Original G-code's extrusion and travel were as following:

	Count	Time (s)	Percentage (%)
Extrusion	383557	5096.988382	83.759565
Travel	80229	988.272905	16.240435

Optimized G-code's

	Count	Time (s)	Percentage (%)
Extrusion	378769	4266.649314	80.253547
Travel	81034	1049.812690	19.746453

Although, extrusion percentage has decreased, extrusion time has decreased in the optimized G-code. Also, the travel time has slightly increased, but overall time has decreased demonstrating the validity of the optimizing algorithm.

G-Code File: 140x200mm_basket_Prusa_0.2mm_PLA_MK3S_13h39m.gcode

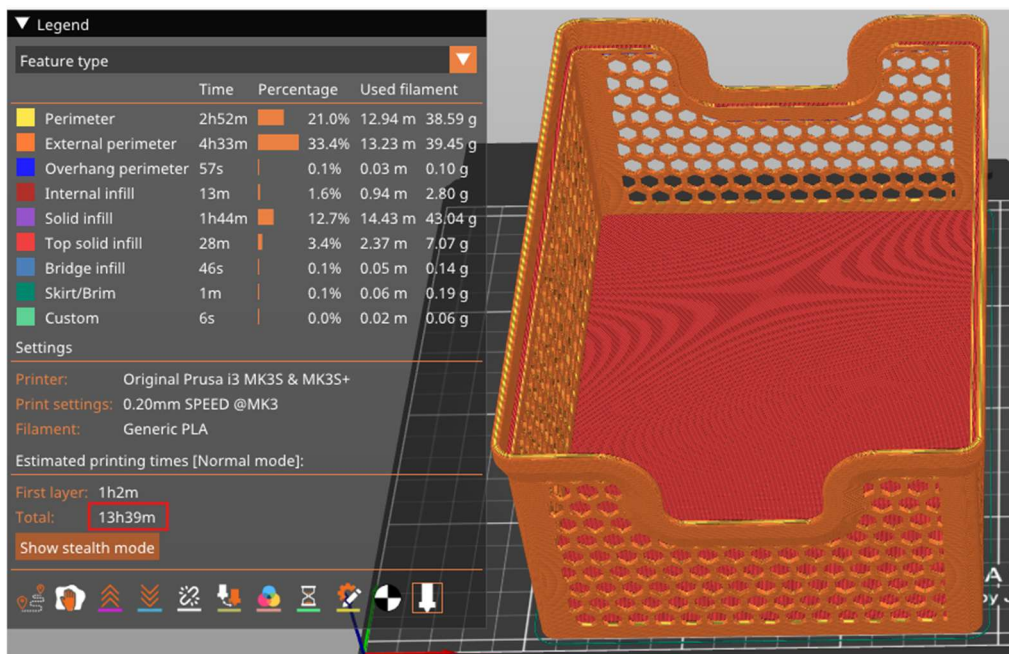


Figure. 11 G-code viewer for original G-code

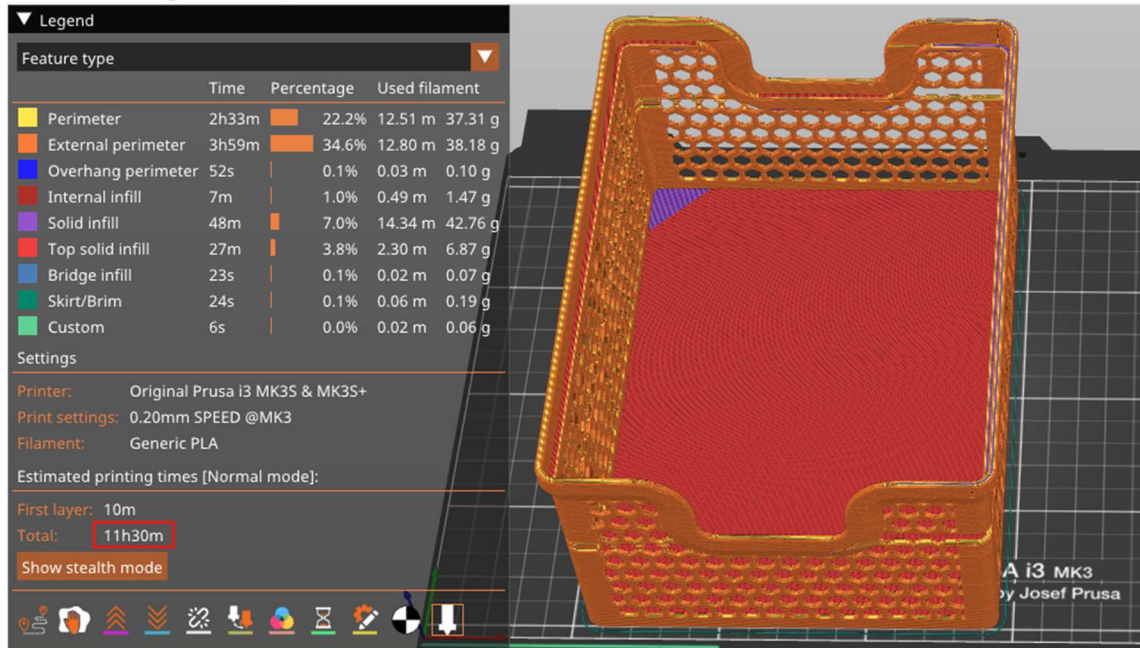


Figure 12: G-code viewer for optimized G-code

Result: Slight decrease in extrusion percentage but extrusion time decreased significantly.

Original G-code's extrusion and travel were as following:

	Count	Time (s)	Percentage (%)
Extrusion	432715	32196.556797	94.982036
Travel	48437	1700.965495	5.017964

Optimized G-code's

	Count	Time (s)	Percentage (%)
Extrusion	400351	25253.798175	93.994314
Travel	50316	1613.569791	6.005686

Although, extrusion percentage has decreased, extrusion time has decreased in the optimized G-code. Also, the travel time has decreased, so overall time has decreased demonstrating the validity of the optimizing algorithm.

G-Code File: tray3_0.2mm_PLA_MK3S_9h33m.gcode

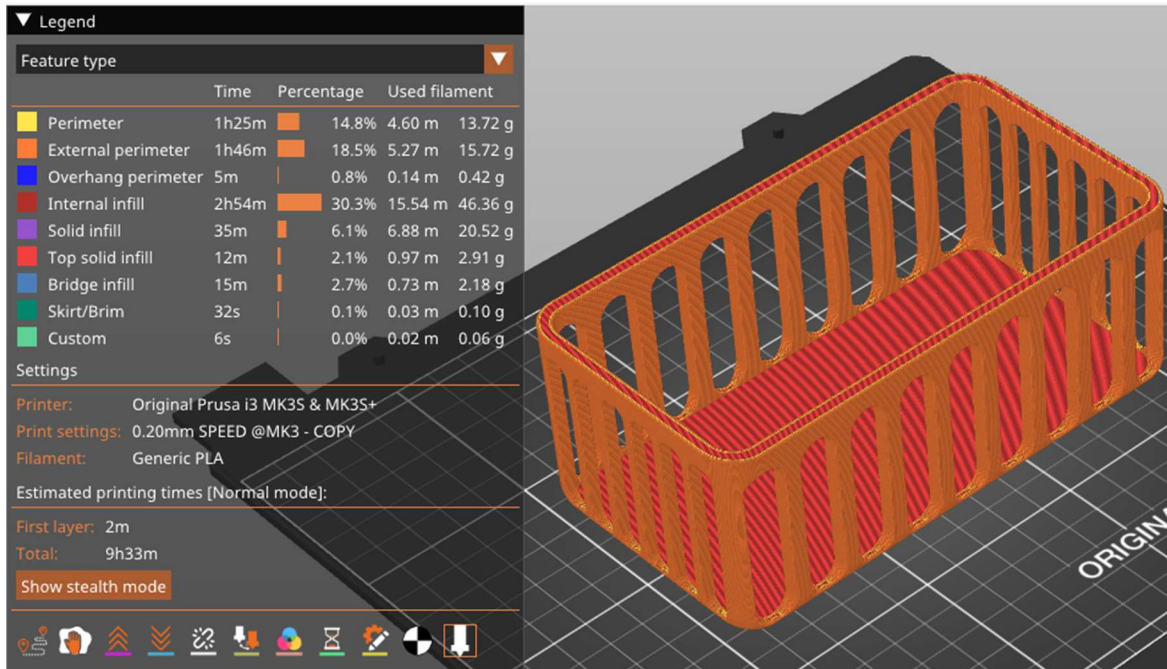


Figure. 13 G-code viewer for original G-code

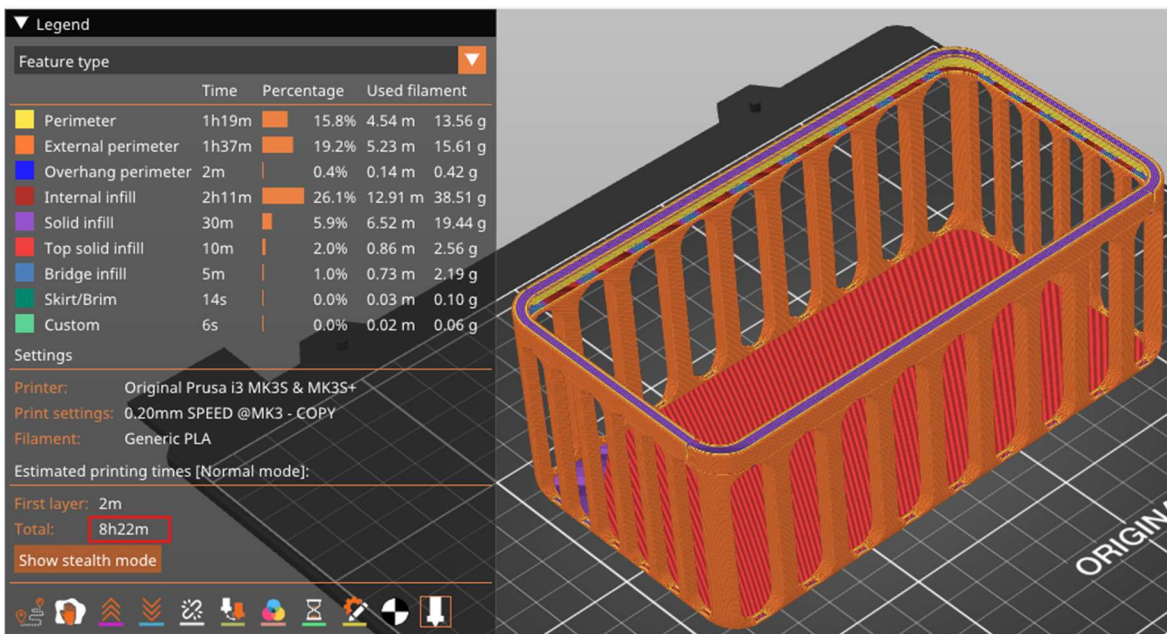


Figure. 14 G-code viewer for optimized G-code

Result: Almost no change in percentage but extrusion time decreased significantly.

Original G-code's extrusion and travel were as following:

	Count	Time (s)	Percentage (%)
Extrusion	416985	16640.463361	94.907113

Travel	26070	892.957349	5.092887
--------	-------	------------	----------

Optimized G-code's

	Count	Time (s)	Percentage (%)
Extrusion	366645	14654.672839	94.406183
Travel	30056	868.328331	5.593817

Extrusion time has decreased in the optimized G-code. Also, the travel time has decreased, so overall time has decreased demonstrating the validity of the optimizing algorithm.

6. Conclusion

Our tool employs a two-stage path-ordering heuristic—first a greedy nearest-neighbor pass that always selects the next-closest extrusion segment, then a 2-Opt local-search refinement to uncross any remaining overlaps. This combined approach consistently reduces non-printing travel and overall print time across a variety of models while using only modest CPU and memory, making it practical for a PC or even an on-board printer controller.

However, for very large G-code files with hundreds or thousands of segments, the optimizer can take several minutes to run, which limits its utility in time-sensitive workflows. Moreover, although our initial goal was to minimize non-printing travel time, the two-stage algorithm often yields only small travel-time savings while sometimes reducing extrusion continuity more aggressively—yet the net effect remains a shorter overall print.

Looking forward, we plan to accelerate the optimization via clustering, more efficient heuristics, or a compiled implementation, and to balance both travel and extrusion improvements. Integrating a true global TSP solver or multi-layer route optimization could also help preserve long extrusion runs, further reduce compute time, and push the print-time savings closer to the theoretical optimum.

7. Acknowledgements

We would like to thank Dr. Wenchao Zhou for sparking our interest in 3D printing software and their applications in Additive Manufacturing. His guidance and enthusiasm laid the foundation for this project. We gratefully acknowledge the designers and maintainers of PrusaSlicer's Printables library, whose freely available CAD models were invaluable for demonstrating and validating our optimization approach.

During the early phase, Youwen conducted a comprehensive literature review of existing path-optimization techniques, while Mehedi designed the architecture and overall workflow that guided the project's development. Salman and Youwen collaborated to identify and evaluate suitable algorithms for the Smart Nozzle Travel Optimization tool.

In the development phase, Youwen sourced and prepared a diverse set of 3D models and generated the corresponding G-code files for testing and optimization. Salman built the core codebase, implemented the two-stage greedy + 2-Opt algorithm, and tested and validated results across multiple models. Mehedi contributed G-code visualization tools and post-processing that was helpful to analyze and refine the optimizer's output.

8. References:

- [1] K.-Y. Fok, C.-T. Cheng, N. Ganganath, H. H.-C. Iu, and C. K. Tse, "Accelerating 3D Printing Process Using an Extended Ant Colony Optimization Algorithm," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*, May 2018, pp. 1–5. doi: 10.1109/ISCAS.2018.8351113.
- [2] X. Lin, Z. Huang, W. Shi, and K. Guo, "A Novel Ant Colony Algorithm for Optimizing 3D Printing Paths," *Electronics*, vol. 13, no. 16, Art. no. 16, Jan. 2024, doi: 10.3390/electronics13163252.
- [3] P. Lechowicz, L. Koszalka, I. Pozniak-Koszalka, and A. Kasprzak, "Path optimization in 3D printer: Algorithms and experimentation system," in *2016 4th International Symposium on Computational and Business Intelligence (ISCBI)*, Sep. 2016, pp. 137–142. doi: 10.1109/ISCBI.2016.7743272.
- [4] Z. Ma, W. Wan, L. Song, C. Liu, H. Liu, and Y. Wu, "An Approach of Path Optimization Algorithm for 3D Concrete Printing Based on Graph Theory," *Applied Sciences*, vol. 12, no. 22, Art. no. 22, Jan. 2022, doi: 10.3390/app122211315.
- [5] K.-Y. Fok, N. Ganganath, C.-T. Cheng, H. H.-C. Iu, and C. K. Tse, "A Nozzle Path Planner for 3-D Printing Applications," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 10, pp. 6313–6323, Oct. 2020, doi: 10.1109/TII.2019.2962241.
- [6] K.-Y. Fok, C.-T. Cheng, and C. K. Tse, "A refinement process for nozzle path planning in 3D printing," in *2017 IEEE International Symposium on Circuits and Systems (ISCAS)*, May 2017, pp. 1–4. doi: 10.1109/ISCAS.2017.8050471.

Appendix:

1. G-code Parser python script

```
import re
import mmap
import pandas as pd
import tkinter as tk
from tkinter import filedialog
import os
import sys

class GcodeParser:
    """
    Encapsulates G-code parsing and statistics.
    """
    def __init__(self, file_path=None):
        self.file_path = file_path or self.select_file()
        self.stats_df = None

    @staticmethod
    def select_file():
        """
        Open a dialog rooted in "./Gcodes" to pick a .gcode file.
        """
        script_dir = os.path.dirname(os.path.abspath(__file__))
        initial_dir = os.path.join(script_dir, 'Gcodes')
        root = tk.Tk()
        root.withdraw()
        path = filedialog.askopenfilename(
            title="Select G-code File",
            initialdir=initial_dir,
            filetypes=[("G-code files", "*.gcode"), ("All files", "*.*")]
        )
        if not path:
            sys.exit("No file selected, exiting.")
        return path

    def parse_stats(self):
        """
        Reads the G-code and computes extrusion/travel counts and times.
        """
        cmd_re = re.compile(r'^(G[0-9]+|M[0-9]+)\b')
        mv_re = re.compile(r'X(?:\d+(\.\d+)?)\s*Y(?:\d+(\.\d+)?)')
```

```

fr_re = re.compile(r'F(\d+(\.\d+)?)')

counts = {'Extrusion Commands': 0, 'Travel Commands': 0}
times = {'Extrusion Time (s)': 0.0, 'Travel Time (s)': 0.0}
pos = {'X': 0.0, 'Y': 0.0}
feed = 0.0

with open(self.file_path, 'r+b') as f:
    mm = mmap.mmap(f.fileno(), 0, access=mmap.ACCESS_READ)
    for raw in iter(mm.readline, b''):
        line = raw.decode('ascii', 'ignore').strip()
        if not line or line.startswith(';'):
            continue
        m = cmd_re.match(line)
        if not m:
            continue
        # update feedrate
        fr = fr_re.search(line)
        if fr:
            feed = float(fr.group(1))
        # determine move
        extrusion = ('E' in line and m.group(1)=='G1')
        mv = mv_re.search(line)
        if mv and feed > 0:
            x, y = float(mv.group(1)), float(mv.group(3))
            dx, dy = x-pos['X'], y-pos['Y']
            dist = (dx*dx + dy*dy)**0.5
            t = dist/feed*60
            key = 'Extrusion Time (s)' if extrusion else 'Travel Time
(s)'

            cnt = 'Extrusion Commands' if extrusion else 'Travel
Commands'

            times[key] += t
            counts[cnt] += 1
            pos['X'], pos['Y'] = x, y
    mm.close()

total = times['Extrusion Time (s)'] + times['Travel Time (s)']
pct_extr = (times['Extrusion Time (s)']/total*100) if total else 0
self.stats_df = pd.DataFrame({
    'Count': [counts['Extrusion Commands'], counts['Travel
Commands']],

```

```

        'Time (s)':      [times['Extrusion Time (s)'], times['Travel Time
(s)']],
        'Percentage (%)':[pct_extr, 100-pct_extr]
    }, index=['Extrusion','Travel'])
    return self.stats_df

def split_file(self):
    """
    Splits the G-code file into header, layers (keyed by Z height), and
    footer.
    Uses Z-move detection. Corrected footer detection logic (more
    conservative).
    """
    print(f"Attempting to split file: {self.file_path}")
    print("INFO: Using Z-move detection for layer splitting.")
    try:
        with open(self.file_path, 'r', encoding='utf-8') as f:
            lines = [l.rstrip('\n') for l in f]
    except UnicodeDecodeError:
        print("Warning: UTF-8 decoding failed, trying latin-1 encoding.")
        with open(self.file_path, 'r', encoding='latin-1') as f:
            lines = [l.rstrip('\n') for l in f]
    except Exception as e:
        print(f"Error reading G-code file in split_file: {e}")
        return [], {}, []

    header, footer = [], []
    layers = {}
    current_layer_z = None # The Z height defining the current layer being
populated
    last_gcode_z = 0.0    # The Z value found in the most recent G0/G1 command
with Z
    found_first_layer_move = False # Flag becomes true after first
significant Z move
    min_layer_z = 0.1 # Ignore Z moves below this threshold as potential
first layer starts

    # --- Regex to find G0/G1 commands and extract Z ---
    g01_command_re = re.compile(r'^([G01])\b(.*)')
    z_param_re = re.compile(r'\bZ(-?\d+(?:\.\d+)?)')

    # --- Definitive Footer Command Markers ---
    # Only commands that strongly indicate the print END sequence

```

```

definitive_footer_markers = ['M104 S0', 'M140 S0', 'M84', 'Turn off
steppers']

print(f"Processing {len(lines)} lines using Z-move layer detection...")

# --- Main Processing Loop (NO footer detection inside) ---
# This loop assigns lines to header or layers dictionary first
for i, line in enumerate(lines):
    # --- Process G0/G1 for Z changes ---
    new_z_this_line = None
    g_match = g01_command_re.match(line)
    if g_match:
        params = g_match.group(2)
        z_match = z_param_re.search(params)
        if z_match:
            try:
                new_z_this_line = float(z_match.group(1))
                # --- Layer Change Logic ---
                if new_z_this_line > (last_gcode_z + 0.001) and
new_z_this_line >= min_layer_z:
                    if current_layer_z is None or abs(new_z_this_line -
current_layer_z) > 0.001:
                        current_layer_z = new_z_this_line
                        if current_layer_z not in layers:
                            layers[current_layer_z] = []
                            print(f" Detected new layer start via Z-
move: Z = {current_layer_z:.3f} at line {i+1}")
                        if not found_first_layer_move:
                            found_first_layer_move = True
                        # Update last_gcode_z only if Z was successfully parsed
                        last_gcode_z = new_z_this_line
                    except ValueError:
                        pass # Ignore if Z value is not parsable

            # --- Line Assignment ---
            if not found_first_layer_move:
                header.append(line)
            elif current_layer_z is not None:
                # Add line temporarily to the current layer dictionary
                # Footer separation happens *after* this loop
                layers[current_layer_z].append(line)

# --- Post-Loop Footer Separation ---

```



```

potential_footer_start_layer_z = None
potential_footer_start_line_index_in_layer = -1
found_definitive_footer_marker = False

# Check only the last few layers or last N lines for efficiency
sorted_layer_keys = sorted(layers.keys(), reverse=True)
lines_to_check_for_footer = 200 # Look within the last ~200 lines added
to layers

checked_line_count = 0
for layer_z in sorted_layer_keys:
    if checked_line_count >= lines_to_check_for_footer:
        break # Stop checking if we've looked back far enough

    layer_lines = layers[layer_z]
    start_index_to_check = max(0, len(layer_lines) -
(lines_to_check_for_footer - checked_line_count))

    # Iterate backwards from end of layer, but only within the check
window
    for idx_in_layer in range(len(layer_lines) - 1, start_index_to_check
- 1, -1):
        line = layer_lines[idx_in_layer]
        checked_line_count += 1

        is_definitive_footer = any(marker in line for marker in
definitive_footer_markers)

        if is_definitive_footer:
            print(f" Definitive footer marker found in layer {layer_z}
at index {idx_in_layer}: {line}")
            # Store the Z and index where the *first* definitive marker
was seen (from the end)
            potential_footer_start_layer_z = layer_z
            potential_footer_start_line_index_in_layer = idx_in_layer
            found_definitive_footer_marker = True
            # Don't break immediately, let it find the earliest marker
within the window if multiple exist

            # If we found a marker in this layer, we don't need to check earlier
layers further back
            # (unless the marker was extremely early in this layer, edge case
ignored for now)

```

```

        # if found_definitive_footer_marker:
        #     break

    # --- Move footer lines from layers to footer list ---
    if found_definitive_footer_marker and potential_footer_start_layer_z is
not None:
        print(f"Moving lines from index
{potential_footer_start_line_index_in_layer} in layer
{potential_footer_start_layer_z} onwards to footer.")
        layer_to_split = layers[potential_footer_start_layer_z]
        footer = layer_to_split[potential_footer_start_line_index_in_layer:]
        layers[potential_footer_start_layer_z] =
layer_to_split[:potential_footer_start_line_index_in_layer]

        # Move lines from any subsequent layers entirely to footer
        # (This shouldn't happen if footer is detected correctly in the last
layer it appears)
        keys_to_delete = []
        for layer_z in sorted(layers.keys()):
            if layer_z > potential_footer_start_layer_z:
                print(f"Moving all lines from subsequent layer {layer_z} to
footer.")
                footer.extend(layers[layer_z])
                keys_to_delete.append(layer_z)
            for key in keys_to_delete:
                del layers[key]
        else:
            print("Warning: No definitive footer markers found near end of file.
Footer might be empty or included in last layer.")
            footer = [] # Assume no separate footer if markers aren't found

    # --- Final Checks ---
    if not layers: print("ERROR: No layers were identified using Z-move
detection.")

    print(f"Split complete (Z-move): Header={len(header)} lines,
Layers={len(layers)}, Footer={len(footer)} lines")
    total_lines_processed = len(header) + sum(len(l) for l in
layers.values()) + len(footer)
    print(f"Total lines processed: {total_lines_processed} / {len(lines)}")
    if total_lines_processed != len(lines): print("WARNING: Line count
mismatch after splitting!")

```

```

        return header, layers, footer
# --- End of replacement for split_file ---

def get_header(self):
    return self.split_file()[0]

def get_layers(self):
    return self.split_file()[1]

def get_footer(self):
    return self.split_file()[2]

def main():
    parser = GcodeParser()
    df = parser.parse_stats()
    print(f"Parsed: {parser.file_path}\n")
    print(df)

if __name__ == "__main__":
    main()

```

2. Smart Nozzle python script - first draft (Greedy Algorithm).

```

import os
import math
from gcode_parser import GcodeParser

class SmartNozzle:
    """
    Applies a greedy nearest-neighbor optimization per layer.
    """
    def __init__(self):
        # Use GcodeParser's file-picker
        self.parser = GcodeParser()
        # These methods assume you've extended GcodeParser to expose:
        # get_header(), get_footer(), get_layers()
        self.header = self.parser.get_header()
        self.footer = self.parser.get_footer()
        self.layers = self.parser.get_layers()
        self.optimized = {}

```

```

@staticmethod
def euclidean(a, b):
    return math.hypot(b[0] - a[0], b[1] - a[1])

def greedy_reorder(self, segments):
    if not segments:
        return []
    remaining = segments.copy()
    current = remaining.pop(0)
    ordered = [current]
    while remaining:
        last = current.end
        nxt = min(remaining, key=lambda s: self.euclidean(last, s.start))
        remaining.remove(nxt)
        ordered.append(nxt)
        current = nxt
    return ordered

def run(self):
    for z, segs in self.layers.items():
        self.optimized[z] = self.greedy_reorder(segs)

def write_output(self):
    out_dir = os.path.join(os.path.dirname(__file__), 'Optimized_Gcodes')
    os.makedirs(out_dir, exist_ok=True)
    base = os.path.basename(self.parser.file_path)
    out_path = os.path.join(out_dir, f"opt_{base}")

    with open(out_path, 'w') as f:
        for line in self.header:
            f.write(line + "\n")
        for z in sorted(self.optimized):
            f.write(f";LAYER:{z}\n")
            for seg in self.optimized[z]:
                for cmd in seg.commands:
                    f.write(cmd + "\n")
        for line in self.footer:
            f.write(line + "\n")

    print(f"Optimized G-code written to:\n {out_path}")

def main():
    optimizer = SmartNozzle()

```

```

optimizer.run()
optimizer.write_output()

if __name__ == "__main__":
    main()

```

3. Smart Nozzle python script - second draft (Greedy Nearest Neighbor Algorithm)

```

import os
import math
from gcode_parser import GcodeParser

class SmartNozzle:
    """
    Parses G-code into travel+extrusion blocks, then applies
    greedy nearest-neighbor on those blocks to optimize travel.
    """

    def __init__(self, parser: GcodeParser):
        self.parser = parser
        with open(self.parser.file_path, 'r') as f:
            self.commands = [line.rstrip('\n') for line in f]
        self.header = []
        self.blocks = []
        self.footer = []
        self.optimized_commands = []
        self.parse_blocks()

    def parse_blocks(self):
        lines = self.commands
        i = 0
        # header: lines before first extrusion
        while i < len(lines):
            line = lines[i].split(';')[0].strip()
            if self.is_extrusion_move(line):
                break
            self.header.append(lines[i])
            i += 1

        # parse blocks
        while i < len(lines):
            # travel block: lines until extrusion
            travel_block = []
            while i < len(lines):

```

```

        line = lines[i].split(';')[0].strip()
        if self.is_extrusion_move(line):
            break
        travel_block.append(lines[i])
        i += 1
    # extrusion block: consecutive extrusion lines
    extrusion_block = []
    while i < len(lines):
        line = lines[i].split(';')[0].strip()
        if not self.is_extrusion_move(line):
            break
        extrusion_block.append(lines[i])
        i += 1
    if extrusion_block:
        self.blocks.append((travel_block, extrusion_block))
    else:
        # no more extrusion, remaining travel is footer
        break

# footer: any remaining lines
self.footer = lines[i:]

def optimize(self):
    remaining = self.blocks.copy()
    ordered_blocks = []
    current_pos = {'X': 0.0, 'Y': 0.0}

    while remaining:
        idx = self.find_nearest_block(current_pos, remaining)
        travel, extr = remaining.pop(idx)
        ordered_blocks.append((travel, extr))
        # update current_pos to end of last extrusion line
        last_line = extr[-1].split(';')[0].strip()
        current_pos = self.get_position_before_move(last_line)

    # rebuild optimized commands list
    self.optimized_commands = []
    for travel, extr in ordered_blocks:
        self.optimized_commands.extend(travel)
        self.optimized_commands.extend(extr)

def write_output(self):
    out_dir = os.path.join(os.path.dirname(__file__), 'Optimized_Gcodes')

```

```

os.makedirs(out_dir, exist_ok=True)
base = os.path.basename(self.parser.file_path)
out_path = os.path.join(out_dir, f"opt_{base}")

with open(out_path, 'w') as f:
    for line in self.header:
        f.write(line + "\n")
    for cmd in self.optimized_commands:
        f.write(cmd + "\n")
    for line in self.footer:
        f.write(line + "\n")

print(f"Optimized G-code written to:\n {out_path}")

def is_extrusion_move(self, line: str) -> bool:
    # Must start with G1 and contain positive E
    if not line.startswith('G1'):
        return False
    for token in line.split():
        if token.startswith('E'):
            try:
                return float(token[1:]) > 0
            except ValueError:
                return False
    return False

def find_nearest_block(self, current_pos: dict, blocks: list) -> int:
    best_idx, best_dist = 0, float('inf')
    for i, (travel, extr) in enumerate(blocks):
        # start of extrusion block
        first_line = extr[0].split(';')[1][0].strip()
        start = self.get_position_before_move(first_line)
        dx = current_pos['X'] - start['X']
        dy = current_pos['Y'] - start['Y']
        d = math.hypot(dx, dy)
        if d < best_dist:
            best_dist, best_idx = d, i
    return best_idx

def get_position_before_move(self, line: str) -> dict:
    # baseline is last known position, but here we treat absolute
    pos = {'X': 0.0, 'Y': 0.0}
    for token in line.split():

```



```

        if token[0] in ('X','Y','Z','E'):
            try:
                pos[token[0]] = float(token[1:])
            except ValueError:
                continue
        return pos

if __name__ == "__main__":
    parser = GcodeParser()
    optimizer = SmartNozzle(parser)
    optimizer.optimize()
    optimizer.write_output()

```

4. Smart Nozzle Optimization Tool python script - (2-Opt TSP optimization)

```

# smart_nozzle.py
import re
import math
import os
import sys
import time

# --- Attempt to import the GcodeParser ---
try:
    from gcode_parser import GcodeParser
except ImportError:
    print("ERROR: Could not import GcodeParser from gcode_parser.py.")
    print("      Ensure gcode_parser.py is in the same directory, is error-  
free,")
    print("      and defines the class 'GcodeParser'.")
    sys.exit(1)
except Exception as e:
    print(f"ERROR: An unexpected error occurred during GcodeParser import: {e}")
    sys.exit(1)

# --- Regular Expressions (Defined at Module Level) ---
G0G1_RE = re.compile(r'^([G01])\b(.*)')
PARAMS_RE = re.compile(r'([XYZEF])(-?\d+(\.\d+)?)')
EXTRUDE_RE = re.compile(r'\bE(-?\d+(\.\d+)?)(!\d)')
FEEDRATE_RE = re.compile(r'\bF(\d+(\.\d+)?)')
Z_PARAM_RE = re.compile(r'\bZ(-?\d+(\.\d+)?)')

```

```

# --- Optimizer Class Definition ---
class GcodeOptimizer:
    """
    Optimizes G-code by reordering extrusion paths within layers
    using 2-Opt TSP heuristic to minimize non-printing travel distance.
    """

    def __init__(self, file_path=None):
        """ Initializes the optimizer, finds global max feedrate from G0/G1. """
        print("Initializing G-code Optimizer...")
        try:
            self.parser = GcodeParser(file_path)
        except NameError:
            print("FATAL ERROR: GcodeParser class definition not found.");
            sys.exit(1)
        except Exception as e:
            print(f"Error initializing GcodeParser: {e}"); sys.exit(1)

        if not hasattr(self.parser, 'file_path') or not self.parser.file_path:
            print("No file path available from GcodeParser. Exiting.");
            sys.exit(0)

        print(f"Selected file: {self.parser.file_path}")
        self.optimized_gcode_lines = []
        self.header = []
        self.layers = {}
        self.footer = []

        # --- Find and store global max feedrate from ANY G0/G1 line ---
        self.global_max_feedrate = 3000.0 # Default fallback if absolutely none
        found
        self._find_global_max_g0_g1_feedrate()

    def _extract_params(self, line):
        # ... (keep implementation) ...
        params = {}
        for match in PARAMS_RE.finditer(line):
            try: params[match.group(1)] = float(match.group(2))
            except (ValueError, IndexError): pass
        return params

```

```

def _calculate_distance(self, pos1, pos2):
    # ... (keep implementation) ...
    if pos1 is None or pos2 is None: return 0.0
    x1 = pos1.get('X', 0.0); y1 = pos1.get('Y', 0.0)
    x2 = pos2.get('X', 0.0); y2 = pos2.get('Y', 0.0)
    if x1 is None or y1 is None or x2 is None or y2 is None: return 0.0
    try:
        dx = float(x1) - float(x2); dy = float(y1) - float(y2)
        return math.sqrt(dx*dx + dy*dy)
    except (TypeError, ValueError): return 0.0

def _find_global_max_g0_g1_feedrate(self):
    """
    MODIFIED: Scans the entire input file once to find the maximum feedrate
    specified on ANY G0 or G1 line.
    """
    print("Scanning file for maximum G0/G1 feedrate...")
    max_feed = 0.0
    found_f = False
    if not hasattr(self, 'parser') or not hasattr(self.parser, 'file_path')
or not self.parser.file_path or not os.path.exists(self.parser.file_path):
        print("Warning: Cannot scan for feedrate, input file path
invalid."); return

    try:
        with open(self.parser.file_path, 'r', encoding='utf-8',
errors='ignore') as f:
            for line in f:
                g_match = G0G1_RE.match(line)
                # Check BOTH G0 and G1 lines
                if g_match:
                    f_match = FEEDRATE_RE.search(g_match.group(2))
                    if f_match:
                        try:
                            feed = float(f_match.group(1))
                            found_f = True
                            if feed > max_feed:
                                max_feed = feed
                        except ValueError: pass # Ignore non-numeric F
    except Exception as e: print(f"Warning: Error reading file during
feedrate scan: {e}")

```

```

        if found_f and max_feed > 0:
            print(f"Found overall maximum G0/G1 feedrate: {max_feed:.2f}")
            self.global_max_feedrate = max_feed
        else:
            print(f"Could not find any F value on G0/G1 lines. Using default:
{self.global_max_feedrate:.2f}")

    def identify_segments(self, layer_z, layer_lines):
        """ Identifies extrusion segments based on extrusion state and
travel/retraction. """
        # ... (Keep the latest refined version of this method) ...
        segments = []
        current_segment_lines = []
        pos_before_segment = {'X': None, 'Y': None, 'Z': layer_z}
        current_abs_pos = {'X': None, 'Y': None, 'Z': layer_z}
        was_extruding = False
        for line_index, line in enumerate(layer_lines):
            g_match = G0G1_RE.match(line)
            params = self._extract_params(line)
            if 'X' in params: current_abs_pos['X'] = params['X']
            if 'Y' in params: current_abs_pos['Y'] = params['Y']
            if 'Z' in params: current_abs_pos['Z'] = params['Z']
            e_match = EXTRUDE_RE.search(line)
            e_value_from_re = float(e_match.group(1)) if e_match else None
            is_extruding_cmd = g_match and g_match.group(1) == 'G1' and
e_value_from_re is not None and e_value_from_re > 1e-5
            is_retracting_cmd = g_match and g_match.group(1) == 'G1' and
e_value_from_re is not None and e_value_from_re < -1e-5
            is_travel_cmd = (g_match and g_match.group(1) == 'G0') or \
(g_match and g_match.group(1) == 'G1' and
e_value_from_re is None and ('X' in params or 'Y' in params))
            should_end_segment = False
            if was_extruding and (is_travel_cmd or is_retracting_cmd):
                should_end_segment = True
            if should_end_segment:
                if current_segment_lines:
                    # --- Finalization Logic ---
                    start_pos_context=dict(pos_before_segment);
seg_start_pos={'X':None,'Y':None}; seg_end_pos={'X':None,'Y':None}

```

```

segment_abs_pos_tracker={'X':start_pos_context.get('X'),'Y':start_pos_context.get('Y')}
        first_move_params=None; has_moves=False;
contains_extrusion=False
        for seg_line in current_segment_lines:
            line_g_match=G0G1_RE.match(seg_line)
            if line_g_match:
                line_params=self._extract_params(seg_line)
                line_e_match=EXTRUDE_RE.search(seg_line);
line_e_val=float(line_e_match.group(1)) if line_e_match else None
                if line_e_val is not None and line_e_val > 1e-5:
contains_extrusion=True
                    is_line_xy_move = 'X' in line_params or 'Y' in
line_params
                        if is_line_xy_move: has_moves=True
                            if 'X' in line_params:
segment_abs_pos_tracker['X']=line_params['X']
                                if 'Y' in line_params:
segment_abs_pos_tracker['Y']=line_params['Y']
                                    if first_move_params is None and is_line_xy_move:
                                        first_move_params=line_params

seg_start_pos['X']=first_move_params.get('X',start_pos_context.get('X'))
seg_start_pos['Y']=first_move_params.get('Y',start_pos_context.get('Y'))
                                if seg_start_pos['X'] is None:
seg_start_pos['X']=0.0
                                    if seg_start_pos['Y'] is None:
seg_start_pos['Y']=0.0
                                        seg_end_pos['X']=segment_abs_pos_tracker['X'];
seg_end_pos['Y']=segment_abs_pos_tracker['Y']
                                            if not has_moves:
                                                seg_start_pos=dict(start_pos_context);
seg_end_pos=dict(start_pos_context)
                                                    if seg_start_pos['X'] is None: seg_start_pos['X']=0.0
                                                        if seg_start_pos['Y'] is None: seg_start_pos['Y']=0.0
                                                            if seg_end_pos['X'] is None: seg_end_pos['X']=0.0
                                                                if seg_end_pos['Y'] is None: seg_end_pos['Y']=0.0
                                                                    # Validation
                                                                        if seg_start_pos.get('X') is not None and
seg_start_pos.get('Y') is not None and \
                                                                            seg_end_pos.get('X') is not None and seg_end_pos.get('Y')
is not None:

```

```

        start_eq_end = abs(seg_start_pos['X'] -
seg_end_pos['X']) < 0.001 and abs(seg_start_pos['Y'] - seg_end_pos['Y']) < 0.001
        if not (start_eq_end and not contains_extrusion):
            segments.append({'lines':
list(current_segment_lines), 'start_pos': dict(seg_start_pos), 'end_pos':
dict(seg_end_pos)})

        # else: Debug print skipping
        # else: print(f"ERROR: Skipping segment (Layer {layer_z:.3f})
due to None start/end XY.")
        current_segment_lines = []
        pos_before_segment = dict(current_abs_pos)
        current_segment_lines.append(line)
        if is_extruding_cmd: was_extruding = True
        elif is_travel_cmd or is_retracting_cmd: was_extruding = False
        # --- Add Last Segment ---
        if current_segment_lines and was_extruding:
            # --- Finalization Logic ---
            start_pos_context=dict(pos_before_segment)
            seg_start_pos = {'X': None, 'Y': None}; seg_end_pos = {'X': None,
'Y': None}
            segment_abs_pos_tracker = {'X': start_pos_context.get('X'), 'Y':
start_pos_context.get('Y')}
            first_move_params = None; has_moves = False; contains_extrusion =
False
            for seg_line in current_segment_lines:
                line_g_match = G0G1_RE.match(seg_line)
                if line_g_match:
                    line_params = self._extract_params(seg_line)
                    line_e_match = EXTRUDE_RE.search(seg_line); line_e_val =
float(line_e_match.group(1)) if line_e_match else None
                    if line_e_val is not None and line_e_val > 1e-5:
contains_extrusion = True
                    is_line_xy_move = 'X' in line_params or 'Y' in line_params
                    if is_line_xy_move: has_moves = True
                    if 'X' in line_params: segment_abs_pos_tracker['X'] =
line_params['X']
                    if 'Y' in line_params: segment_abs_pos_tracker['Y'] =
line_params['Y']
                    if first_move_params is None and is_line_xy_move:
                        first_move_params = line_params
                        seg_start_pos['X'] = first_move_params.get('X',
start_pos_context.get('X'))
                        seg_start_pos['Y'] = first_move_params.get('Y',
start_pos_context.get('Y'))

```

```

        if seg_start_pos['X'] is None: seg_start_pos['X'] = 0.0
        if seg_start_pos['Y'] is None: seg_start_pos['Y'] = 0.0
        seg_end_pos['X'] = segment_abs_pos_tracker['X']; seg_end_pos['Y'] =
segment_abs_pos_tracker['Y']
        if not has_moves:
            seg_start_pos = dict(start_pos_context); seg_end_pos =
dict(start_pos_context)
            if seg_start_pos['X'] is None: seg_start_pos['X'] = 0.0
            if seg_start_pos['Y'] is None: seg_start_pos['Y'] = 0.0
            if seg_end_pos['X'] is None: seg_end_pos['X'] = 0.0
            if seg_end_pos['Y'] is None: seg_end_pos['Y'] = 0.0
        # Validation
        if seg_start_pos.get('X') is not None and seg_start_pos.get('Y') is
not None and \
            seg_end_pos.get('X') is not None and seg_end_pos.get('Y') is not
None:
            start_eq_end = abs(seg_start_pos['X'] - seg_end_pos['X']) <
0.001 and abs(seg_start_pos['Y'] - seg_end_pos['Y']) < 0.001
            if not (start_eq_end and not contains_extrusion):
                segments.append({'lines':
list(current_segment_lines), 'start_pos': dict(seg_start_pos), 'end_pos':
dict(seg_end_pos)})
            # else: Debug print skipping
            # else: print(f"ERROR: Skipping LAST segment (Layer {layer_z:.3f})
due to None start/end XY.")
            # --- End Finalization ---

    return segments

def optimize_layer(self, layer_z, layer_lines):
    """ Optimizes travel moves using 2-Opt TSP and global max G0/G1 feedrate.
    """
    segments = self.identify_segments(layer_z, layer_lines)
    if not segments or len(segments) <= 1: return layer_lines
    num_segments = len(segments)

    # --- Use the globally stored MAX G0/G1 feedrate for travel ---
    travel_feedrate = self.global_max_feedrate
    # ---

    # --- Helper function for path distance ---
    def calculate_path_distance(order, segments):

```

```

        # ... (Keep implementation with safety checks) ...
        total_dist = 0.0
        for k in range(num_segments - 1):
            idx1, idx2 = order[k], order[k+1]
            if not (0 <= idx1 < len(segments) and 0 <= idx2 < len(segments)):
                return float('inf')
            pos1, pos2 = segments[idx1].get('end_pos'),
            segments[idx2].get('start_pos')
            if pos1 is None or pos2 is None or \
                pos1.get('X') is None or pos1.get('Y') is None or \
                pos2.get('X') is None or pos2.get('Y') is None: return
            float('inf')
            total_dist += self._calculate_distance(pos1, pos2)
        return total_dist

    # --- 2-Opt Algorithm ---
    current_order = list(range(num_segments))
    initial_distance = calculate_path_distance(current_order, segments)
    if initial_distance == float('inf'): return layer_lines

    best_order = current_order[:]
    best_distance = initial_distance

    improved = True; iteration_count = 0; max_iterations = max(500,
num_segments * 10)
    while improved and iteration_count < max_iterations:
        iteration_count += 1; improved = False
        for i in range(num_segments - 1):
            for j in range(i + 1, num_segments):
                new_order = best_order[:]; sub_path = new_order[i+1 : j+1]
                sub_path.reverse(); new_order[i+1 : j+1] = sub_path
                new_distance = calculate_path_distance(new_order, segments)
                if new_distance < (best_distance - 1e-3):
                    best_order = new_order; best_distance = new_distance;
improved = True
                break # Use first improvement strategy
            if improved: break
    # --- End 2-Opt ---

    # --- Reconstruct G-code ---
    optimized_gcode = []
    if not best_order: return layer_lines

```



```

first_segment_index = best_order[0]
if not (0 <= first_segment_index < len(segments)): return layer_lines
optimized_gcode.extend(segments[first_segment_index]['lines'])
last_pos = segments[first_segment_index].get('end_pos')
if last_pos is None or last_pos.get('X') is None or last_pos.get('Y') is
None: return layer_lines

for k in range(1, num_segments):
    current_segment_index = best_order[k]
    if not (0 <= current_segment_index < len(segments)): break
    next_seg = segments[current_segment_index]
    start_pos = next_seg.get('start_pos')
    if start_pos is None or start_pos.get('X') is None or
start_pos.get('Y') is None: continue

    start_x, start_y = start_pos['X'], start_pos['Y']
    if self._calculate_distance(last_pos, start_pos) > 1e-3:
        # --- Use the determined max G0/G1 feedrate for generated G0 ---
        travel_move = f"G0 F{travel_feedrate:.2f} X{start_x:.4f}
Y{start_y:.4f}"
        optimized_gcode.append(travel_move)

    optimized_gcode.extend(next_seg['lines'])
    current_end_pos = next_seg.get('end_pos')
    if current_end_pos is not None and current_end_pos.get('X') is not
None and current_end_pos.get('Y') is not None:
        last_pos = current_end_pos

return optimized_gcode

def optimize_gcode(self):
    """ Parses G-code, optimizes layers, reconstructs file content. """
    start_time = time.time()
    print("Starting G-code optimization process...")
    try:
        self.header, self.layers, self.footer = self.parser.split_file()
        if not isinstance(self.layers, dict) or not self.layers:
            print("Error: GcodeParser did not return valid layers
dictionary."); return False
        print(f"Found {len(self.layers)} layers.")
    except AttributeError: print("Error: Problem calling GcodeParser's
split_file method."); return False

```

```

        except Exception as e: print(f"Error during parsing/splitting: {e}");
return False

    optimized_layers_content = []
    try: sorted_layer_keys = sorted(self.layers.keys())
    except TypeError: print("Error: Layer keys are not sortable."); return
False

    layer_count_processed = 0
    layer_count_optimized = 0 # Count layers where 2-opt actually ran
    for layer_z in sorted_layer_keys:
        layer_lines = self.layers.get(layer_z, [])
        if not layer_lines: continue
        layer_count_processed += 1
        optimized_layers_content.append(f"; OPTIMIZED_LAYER_START
Z={layer_z:.3f}")
        optimized_lines = self.optimize_layer(layer_z, layer_lines)
        # Check if optimization actually ran (more than 1 segment existed)
        if len(optimized_lines) != len(layer_lines): # Crude check: Did lines
change?
            layer_count_optimized +=1
            optimized_layers_content.extend(optimized_lines)

    self.optimized_gcode_lines = self.header + optimized_layers_content +
self.footer
    end_time = time.time()
    print(f"Layer optimization loop finished processing
{layer_count_processed} layers in {end_time - start_time:.2f} seconds.")
    if layer_count_optimized > 0:
        print(f"Optimization potentially applied to {layer_count_optimized}
layers (those with >1 segment).")
    else:
        print("No layers had multiple segments suitable for optimization.")

    return True

def save_optimized_gcode(self):
    """ Saves the optimized G-code automatically to 'Optimized_Gcodes'
sibling folder. """
    # ... (Keep the latest automatic save version without dialog) ...
    if not self.optimized_gcode_lines: print("No optimized G-code generated
to save."); return

```

```

        if not hasattr(self, 'parser') or not hasattr(self.parser, 'file_path')
or not self.parser.file_path:
            print("Error: Cannot determine input file path for saving."); return

        try:
            input_file_path = self.parser.file_path; input_dir =
os.path.dirname(input_file_path)
            parent_dir = os.path.dirname(input_dir); output_dir =
os.path.join(parent_dir, "Optimized_Gcodes")
            original_filename = os.path.basename(input_file_path)
            optimized_filename = os.path.splitext(original_filename)[0] +
"_optimized.gcode"
            output_file_path = os.path.join(output_dir, optimized_filename)

            if not os.path.exists(output_dir):
                try: print(f"Creating output directory: {output_dir}");
os.makedirs(output_dir)
                    except OSError as e: print(f"Error creating output directory
{output_dir}: {e}"); print(f"Fallback: Saving to parent directory:
{parent_dir}"); output_file_path = os.path.join(parent_dir, optimized_filename)
                    except Exception as e_inner: print(f"Unexpected error creating
directory {output_dir}: {e_inner}"); print(f"Fallback: Saving to parent
directory: {parent_dir}"); output_file_path = os.path.join(parent_dir,
optimized_filename)

                print(f"Attempting to automatically save optimized file to:
{output_file_path}")
                with open(output_file_path, 'w', encoding='utf-8') as f:
f.write("\n".join(self.optimized_gcode_lines))
                print(f"Optimized G-code saved successfully.")

            except AttributeError: print("Error: Missing parser information needed
for saving.")
            except Exception as e: print(f"Error during automatic save operation:
{e}"); import traceback; traceback.print_exc()

# --- Main Function (Defined at Module Level) ---
def main():
    """ Main execution function: Creates optimizer and runs the process. """
    try:
        optimizer = GcodeOptimizer()
        if optimizer.optimize_gcode():

```

```

        optimizer.save_optimized_gcode()
    else:
        print("\nOptimization process failed or aborted.")
except ValueError as ve:
    print(f"Initialization failed: {ve}")
except NameError as ne:
    print(f"NameError: {ne}. Is GcodeOptimizer class defined correctly?")
except Exception as e:
    print(f"\nAn unexpected error occurred in main execution: {e}")
    import traceback
    traceback.print_exc()

# --- Script Execution Guard (Module Level) ---
if __name__ == "__main__":
    # This ensures main() runs only when script is executed directly
    main()

```